

编译原理第四章—语义分析考点总结

课件整理：语法制导翻译（42 页）

2026 年 5 月 12 日

目录

1	语法制导翻译概述	2
2	语法树的生成	3
3	语法制导翻译的理论基础	5
4	语义分析扩展要点	7
5	核心考点速记	8

1 语法制导翻译概述

1.1 什么是语法制导翻译 (Syntax-Directed Translation, SDT)

- 语法制导翻译是语法分析指导下的翻译：在语法分析过程中，给每条产生式规则附加一个翻译动作 [p.7-8]
- 翻译动作分为两类 [p.8, 16]:
 - (1) 生成动作：生成语法树、生成中间代码或目标代码
 - (2) 语义动作：赋予类型、检查类型
- SDT 可以用于生成语法树、语义分析（如类型检查）和生成中间/目标代码 [p.7]

1.2 前端内部的阶段与语义分析的位置

- 编译器前端流程：源代码 → 词法分析器 → 记号流 → 语法分析器 → 语法树 → 语义分析器 → 中间表示 [p.5]
- 语义分析器的输入是语法分析器输出的语法树，负责类型检查、作用域分析等任务 [p.5-6]
- 语法制导翻译是连接前后端的核心机制 [p.40]

1.3 语法制导翻译的基本原理

- 语法分析在使用某条产生式时插入翻译动作 [p.13]:
 - 自顶向下分析：将产生式展开成右部时插入动作
 - 自底向上分析：将产生式右部收缩（归约）成左部时插入动作
- SDT 示意（自底向上 LR 示例：消除左递归后计算表达式） [p.13]:

```
E  -> n E'      {E.val = n.val + E'.val}
E' -> +E E'      {E'.val = E.val + E'.val}
E' -> \varepsilon {E'.val = 0}
```

1.4 LR 分析中的语法制导翻译实现

- 在分析栈上维护三元组：〈状态, 记号, 属性〉 [p.14]
- 状态栈 + 记号栈构成语法分析栈，属性栈存储栈顶记号对应的属性值 [p.14-15]
- 完整示例：分析输入串 $7 + 8$ ，使用文法 $E \rightarrow E + E \mid n$ [p.15]:
 - 规约 $E \rightarrow n$ 时执行语义动作 $\{\$ \$ = \$ 1\}$ ，将记号值写入属性栈
 - 规约 $E \rightarrow E + E$ 时执行语义动作 $\{\$ \$ = \$ 1 + \$ 3\}$ ，计算属性值

2 语法树的生成

2.1 语法树的基本概念

- 语法树用图形方式展现从文法的开始符号推导出符号串的过程 [p.18]
- 特点：内部节点代表非终结符，叶子节点代表终结符，每一步推导代表父节点生成直接孩子节点 [p.18]
- 语法树与程序构造的对应：父节点为程序构造的动作，子节点为执行对象 [p.19]

2.2 语法树的数据结构 (C 语言示例)

- 语法树数据结构与实现语言密切相关 [p.21]
- 早期编译器可能不采用显式语法树，直接在 SDT 中生成代码；现代编译器普遍采用语法树作为语法分析器输出，以获得更好的系统支持和简化设计 [p.21]
- C 语言版语法树节点定义 [p.22-23]:

```
typedef struct Exp *Exp_t;
struct Exp {
    enum {E_int, E_add, E_times} kind;
    union { int n;
           struct { Exp_t left, right; } e;
    };
};
```

- 构造函数定义 [p.23]:

```
Exp_t Exp_Int_new(int n);
Exp_t Exp_Add_new(Exp_t left, Exp_t right);
Exp_t Exp_Times_new(Exp_t left, Exp_t right);
```

2.3 利用 SDT 自动生成语法树

- 在语法动作中加入生成语法树的代码片段（构造函数调用） [p.25]:

```
E -> n      { e = Exp_Int_new(n);      }
E -> E + E   { e = Exp_Add_new(e1, e2); }
E -> E * E   { e = Exp_Times_new(e1, e2); }
```

- LR 归约时自底向上构建整棵树（从叶子到根节点） [p.26]
- 示例：输入 $2 + 3 * 4$ ，规约顺序为 $n \rightarrow E, n \rightarrow E, n \rightarrow E, E * E \rightarrow E, E + E \rightarrow E$ ，每次归约都调用对应构造函数 [p.26]

2.4 源代码信息的保留

- 语法树是编译器前端和后端的**接口**，一旦转换成语法树，源代码即被丢弃，后续阶段只处理语法树 [p.27]
- 语法树必须编码足够多的源代码信息，特别是每个语法结构在源代码中的**位置信息**（文件、行号、列号），以便后续阶段精确报错 [p.27–28]
- 语法树的设计至关重要，需要仔细规划 [p.27]

3 语法制导翻译的理论基础

3.1 语法制导定义 (Syntax-Directed Definition, SDD)

- **SDD**: 一个上下文无关文法, 其中每个文法符号具备一个属性, 且每条产生式具备一个语义动作 [p.31]
- **属性文法 (Attribute Grammar)**: 没有副作用的 SDD 文法 [p.31]
- **副作用示例**: 在语义动作中使用 `printf()` 输出即产生副作用 [p.31]

产生式	语义规则
$E \rightarrow E_1 + E_2$	$\{E.val = E_1.val + E_2.val\}$
$E \rightarrow n$	$\{E.val = n.val\}$

3.2 SDD 的分类

类型	可否在语法分析中完成	特征
S 属性的 SDD	可以 [p.35]	所有属性都是综合属性
L 属性的 SDD	可以 [p.36–37]	继承属性受限于指定规则
其他 SDD	不能 [p.38]	存在非 S 非 L 的依赖

3.3 综合属性 (Synthesized Attribute / S 属性)

- **定义**: 节点上的综合属性只能通过其子节点或者节点本身的属性来定义 [p.33]
- **S 属性的 SDD**: 每个属性都是综合属性的 SDD, 只存在自底向上的属性依赖关系 [p.33]
- **直观理解**: 在语法树中, 综合属性从叶子向根传播, 先计算子树属性再计算父节点属性 [p.33]
- **实现**: 自底向上语法分析中, 在产生式右部规约为左部时执行翻译动作 (so easy!) [p.35]
- **典型示例**: 表达式求值中的 val 属性:

```

E -> E + T { E.val = E1.val + T.val }
T -> F      { T.val = F.val }
F -> num    { F.val = num.lexval }

```

3.4 继承属性 (Inherited Attribute / I 属性)

- **定义**: 节点上的继承属性只能通过其父节点、节点本身或者兄弟节点的属性来定义 [p.34]
- **L 属性的 SDD**: 每个属性都是综合属性, 或者满足如下规则的继承属性 [p.34]:
 - (1) 由父节点的属性定义
 - (2) 由左兄弟的属性定义
 - (3) 由本身的其他属性定义

- **核心约束**：不能存在依赖环；只存在自底向上/自顶向下/自左向右的依赖，不存在自右向左的依赖 [p.34]
- **典型示例**：变量声明的类型传递 [p.34]：

```

D  -> T L      { L.type = T.type }
T  -> int      { T.type = "integer" }
T  -> float    { T.type = "float" }
L  -> id , L1   { id.type = L.type; L1.type = L.type }
L  -> \varepsilon { }

```

语法树中，type 属性从 T 节点流到 L 节点，再从左流向右（通过兄弟节点链），属于典型的 L 属性依赖 [p.34]

3.5 依赖图 (Dependency Graph) 与注释语法树

- **依赖图**：描述语法树中各属性之间的依赖关系的有向图 [p.38]
 - 节点：属性实例（语法树中每个文法符号的每个属性）
 - 边：从依赖源指向被依赖目标（表示计算顺序）
- **注释语法树 (Annotated Parse Tree)**：在语法树节点上标注了属性值的树 [p.33]
- 对于非 S 非 L 的 SDD，需要先建语法树，画依赖图，判断无环后才能拓扑排序求解属性值 [p.38]

3.6 L 属性 SDD 的实现策略

- **递归下降分析** [p.36]：
 - 非终结符 N 的**继承属性**作为子分析函数的**参数**传入
 - 非终结符 N 的**综合属性**作为子分析函数的**返回值**返回
 - 左边的子函数先执行，所以左依赖的属性值先计算，天然符合 L 属性要求
- **自底向上分析** [p.37]：也可以在 LR 分析中实现 L 属性 SDD，但较复杂（课堂超纲，参见龙书 5.5.4 节）

3.7 非 S 非 L 属性的 SDD 处理

- **黄金法则**：实际编译器中不要设计非 S 非 L 的 SDD [p.38]
- 如果真有这样的 SDD，通用方法 [p.38]：
 - (1) 先分析一遍语法，建立语法分析树
 - (2) 画出属性之间的依赖图，确保不存在环
 - (3) 对依赖关系进行拓扑排序
 - (4) 按照依赖关系顺序求解属性值
- **实际工程观点** [p.39]：通用的形式化属性语法方法在实践中从未得到广泛使用——求解复杂、需要遍历语法树、临时空间管理繁琐。实际中通常手写满足 S 属性或 L 属性的翻译动作

4 语义分析扩展要点

4.1 类型系统 (Type System)

- **类型表达式 (Type Expression)**: 用于描述类型的数学表示, 包括基本类型 (int, float, char)、数组类型、函数类型、指针类型、结构体类型等
- **类型等价 (Type Equivalence)** 的两种定义:
 - (1) **结构等价 (Structural Equivalence)**: 两个类型当且仅当它们具有相同的结构时等价
 - (2) **名字等价 (Name Equivalence)**: 每个类型定义引入一个新类型, 即使结构相同, 名字不同即不等价
- 类型表达式的构造——从声明语句推导类型表达式, 与属性文法密切相关 (继承属性传递类型信息, 如前面 $D \rightarrow T L$ 的例子 [p.34])

4.2 类型检查 (Type Checking)

- 类型检查是语义分析的核心任务, 验证程序中操作符/函数是否应用于正确类型的操作数/参数
- 通过语法制导定义实现: 给表达式非终结符附加 **type** 属性, 在归约时进行类型一致性验证
- 类型转换 (隐式/显式) 可在语义动作中插入转换节点

4.3 符号表 (Symbol Table)

- **符号表**: 存储标识符信息的核心数据结构, 每条记录包含标识符名字、类型、作用域级别、存储类别等信息
- 常用操作: `insert(name, info)` 插入条目、`lookup(name)` 查找条目
- **作用域规则 (Scope Rules)**:
 - **静态作用域 (Static/Lexical Scope)**: 标识符的绑定由程序的词法结构决定 (编译期确定)
 - **最近嵌套原则**: 查找从最内层作用域开始, 逐级向外
- 符号表常用实现: 哈希表 + 作用域栈 (进入新作用域时压栈, 离开时弹出并释放该作用域的符号)

5 核心考点速记

1. SDT vs SDD vs 属性文法 [p.7, 31]:

SDT 是语法分析指导下的翻译总称; SDD 是 SDT 的形式化描述 (文法符号 + 属性 + 语义动作); 属性文法是无副作用的 SDD。

2. 综合属性 vs 继承属性 [p.33-34]:

综合属性 = 从子节点/自身计算 (自底向上); 继承属性 = 从父节点/兄弟节点/自身计算 (可能自顶向下或自左向右)。

3. S 属性 SDD [p.33, 35]: 所有属性都是综合属性, 适合自底向上 LR 分析, 归约时计算。

4. L 属性 SDD [p.34]: 属性的依赖只涉及父节点/左兄弟/自身, 不能有右兄弟依赖 (即依赖图无自右向左的边), 适合递归下降分析 (继承属性 = 参数, 综合属性 = 返回值)。

5. 依赖图与拓扑排序 [p.38]: 非 S 非 L 的 SDD 需要建语法树, 画依赖图, 检查无环后拓扑排序求解。实践中应避免设计此类 SDD。

6. 属性栈 (LR 分析) [p.14-15]: 自底向上 LR 分析的 SDT 实现核心是在状态栈 + 记号栈之外增加属性栈, 维护三元组 (状态, 记号, 属性)。归约时弹出右部符号的属性, 计算左部符号的属性并压入。

7. 语法树自动生成 [p.25-26]: 通过在文法产生式的语义动作中调用构造函数实现。LR 分析自底向上从叶子到根构建。语法树必须保留源代码位置信息 (文件/行号/列号) 以支持精确报错 [p.27]。

8. 类型检查与符号表 [p.7, 34]: 类型检查是语义分析的核心任务。符号表中的类型信息通过继承属性在语法树中传播 (如变量声明的 $D \rightarrow T L$ 模式); 综合属性用于表达式的类型推导。

9. 类型等价: 结构等价比较类型结构, 名字等价比较类型名称。两种策略影响类型检查的严格程度。

10. 符号表作用域: 静态作用域 + 最近嵌套原则。实现通常采用哈希表 + 作用域栈, 进入/退出作用域时压栈/弹栈。

考试重点提醒

判断属性类别 (综合/继承) 是必考题 [p.41];

理解 S 属性和 L 属性的区别与实现方法 [p.33-37];

能够为给定翻译目标设计 SDD 并写出语义规则 [p.42]。